Module Guide for FitFolio

Team #1, Burn n' Earn
Davis Lenover
Preston Grewal
Angad Chhabra
Farid Bastoros
Zaid Alsaadi

November 8, 2026

1 Revision History

Date	Version	Notes
Nov 5, 2026	1.0	Davis Lenover: Revision −1. Sections 1 to 3, 5 Module Hierarchy, and 9 Use Hierarchy Between Modules.

2 Reference Material

This section records information for easy reference.

2.1 Abbreviations and Acronyms

symbol	description
AC	Anticipated Change
API	Application Programming Interface
DAG	Directed Acyclic Graph
FitFolio	A phone app where physical activity earns virtual currency that the user invests in a simulated stock market
FR	Functional Requirement (SRS)
HA	Hazard Analysis
JSON	JavaScript Object Notation
LC	Likely Change (SRS)
M	Module
MG	Module Guide
MIS	Module Interface Specification
NFR	Nonfunctional Requirement (SRS)
OS	Operating System
PoC	Proof of Concept
REST	Representational State Transfer
SRS	Software Requirements Specification
UC	Unlikely Change

Contents

1 Revision History **i**

2 Reference Material **ii**

2.1 Abbreviations and Acronyms ii

3 Introduction **1**

4 Anticipated and Unlikely Changes **2**

4.1 Anticipated Changes 2

4.2 Unlikely Changes 3

5 Module Hierarchy **3**

6 Connection Between Requirements and Design **4**

7 Module Decomposition **5**

7.1 Hardware Hiding Modules 6

7.1.1 Database Module (M1) 6

7.1.2 Clock Module (M2) 6

7.1.3 Market Providers Module (M3) 6

7.2 Behaviour-Hiding Modules 7

7.2.1 Control Module (M6) 7

7.2.2 Constants Module (M7) 7

7.2.3 Market Module (M10) 7

7.2.4 Trading Module (M12) 8

7.3 Software Decision Modules 8

7.3.1 Plugins Module (M22) 8

7.3.2 JSON Format Module (M23) 9

8 Traceability Matrix **9**

9 Use Hierarchy Between Modules **10**

10 User Interfaces **12**

11 Design of Communication Protocols **13**

12 Database Design **13**

13 Timeline **13**

14 Generative AI Use Disclosure **13**
14.1 AI Tools Used . . . 13
14.2 How and Where Used . . . 13
14.3 Verification of AI-Generated Content . . . 13

List of Tables

1 Module Hierarchy . . . 5
2 Trace Between Requirements and Modules . . . 9
3 Trace Between Anticipated Changes and Modules . . . 10
4 Modules each module uses . . . 12

List of Figures

1 Use hierarchy among modules. Blue: hardware-hiding. Purple: behaviour-hiding. Orange: software decision. A dashed outline marks a module built after the proof of concept. . . . 11

3 Introduction

FitFolio joins two things that are usually kept apart: fitness and investing. Activity recorded on the phone earns virtual currency, the user spends it on simulated stocks priced from a real market, and the results feed goals, learning content and a social feed. That brings three design challenges:

- The features depend on each other. A user buying a stock needs the latest price from the market and their balance from the wallet, and must change both the balance and the shares together. Activity rewards land in the same wallet. The social feed shares activity and milestones, but must never show a balance or health data.
- Some parts rely on outside services that may change: the market data provider, which allows users to buy shares in the first place, and the health platforms on iOS and Android.
- Given this is a group project, the system will be developed in parallel, in stages, from the proof of concept to Revision 1.

When we first designed the backend, we routed commands to different services and let those services talk to each other directly, each with its own libraries. Taking a step back, this greatly increased coupling: a logic change in any one of these modules could easily ripple through the others. We needed a way to separate the modules, so that each one keeps its data and its decisions to itself, and the others reach it only through a small interface. This helps to support changes and additions to the modules.

Decomposing a system into modules is a commonly accepted approach to developing software. We decompose FitFolio by information hiding: each module hides a "secret", a design decision that is likely to change, such as the market data provider or a health platform. Separate modules also naturally become work assignments that people can develop in tandem. With a defined interface in between, it becomes much easier to work in parallel.

FitFolio is two programs: a server, written in Python with FastAPI and backed by PostgreSQL, and a phone app, written in React Native. The phone app talks to the server over HTTPS with JSON. This MG covers the modules of both programs.

Our design follows three rules:

- System details that are likely to change independently should be the secrets of separate modules.
- Each data structure is implemented in only one module.
- Any other program that requires information stored in a module's data structures must obtain it by calling access programs belonging to that module.

After completing the first stage of the design, the Software Requirements Specification (SRS), the Module Guide (MG) is developed. The MG specifies the modular structure of the system and is intended to allow both designers and maintainers to easily identify the parts of the software. The potential readers of this document are as follows:

- New project members: This document can be a guide for a new project member to easily understand the overall structure and quickly find the relevant modules they are searching for.
- Maintainers: The hierarchical structure of the module guide improves the maintainers' understanding when they need to make changes to the system. It is important for a maintainer to update the relevant sections of the document after changes have been made.
- Designers: Once the module guide has been written, it can be used to check for consistency, feasibility, and flexibility. Designers can verify the system in various ways, such as consistency among modules, feasibility of the decomposition, and flexibility of the design.

The rest of the document is organized as follows. Section 4 lists the anticipated and unlikely changes of the software requirements. Section 5 summarizes the module decomposition that was constructed according to the likely changes. Section 6 specifies the connections between the software requirements and the modules. Section 7 gives a detailed description of the modules. Section 8 includes two traceability matrices. One checks the completeness of the design against the requirements provided in the SRS. The other shows the relation between anticipated changes and the modules. Section 9 describes the use relation between modules. The remaining sections cover the user interfaces, the communication protocols, the database design and the timeline.

4 Anticipated and Unlikely Changes

This section lists possible changes to the system. According to the likeliness of the change, the possible changes are classified into two categories. Anticipated changes are listed in Section 4.1, and unlikely changes are listed in Section 4.2.

4.1 Anticipated Changes

Anticipated changes are the source of the information that is to be hidden inside the modules. Ideally, changing one of the anticipated changes will only require changing the one module that hides the associated decision. The approach adapted here is called design for change.

AC1: The specific hardware on which the software is running.

AC2: The format of the initial input data.

...

[Anticipated changes relate to changes that would be made in requirements, design or implementation choices. They are not related to changes that are made at run-time, like the values of parameters. —SS]

4.2 Unlikely Changes

The module design should be as general as possible. However, a general system is more complex. Sometimes this complexity is not necessary. Fixing some design decisions at the system architecture stage can simplify the software design. If these decision should later need to be changed, then many parts of the design will potentially need to be modified. Hence, it is not intended that these decisions will be changed.

UC1: Input/Output devices (Input: File and/or Keyboard, Output: File, Memory, and/or Screen).

...

5 Module Hierarchy

This section provides an overview of the module design. Modules are summarized in a hierarchy decomposed by secrets in Table 1. The modules listed below, which are leaves in the hierarchy tree, are the modules that will actually be implemented.

The hierarchy covers both programs in FitFolio, the server and the phone app. Modules that are only in the app are marked (app). Hardware-hiding modules hide the interfaces of software and devices the team does not write. These are the database server, the server's clock, the market data provider, and the phone's health platform and operating system. Behaviour-hiding modules hide the behaviour the SRS describes. Software decision modules hide choices the team made that the SRS does not mention.

M1: Database

M2: Clock

M3: Market Providers

M4: Health Service (app)

M5: Device Service (app)

M6: Control

M7: Constants (server and app)

M8: Users

M9: Wallet

M10: Market

M11: Activity

M12: Trading

M13: Goals†

M14: Learning†

M15: Social†

M16: Scenarios†

M17: Privacy†

M18: Navigation (app)

M19: Screens (app)

M20: Sync Service (app)

M21: Reminder Service (app)†

M22: Plugins

M23: JSON Format

M24: API Service (app)

Modules marked with † in Table 1 are built after the proof of concept and are specified in the MIS at Revision 0.

6 Connection Between Requirements and Design

The design of the system is intended to satisfy the requirements developed in the SRS. In this stage, the system is decomposed into modules. The connection between requirements and modules is listed in Table 2.

[The intention of this section is to document decisions that are made "between" the requirements and the design. To satisfy some requirements, design decisions need to be made. Rather than make these decisions implicit, they are explicitly recorded here. For instance, if a program has security requirements, a specific design decision may be made to satisfy those requirements with a password. —SS]

Level 1	Level 2
Hardware-Hiding Module	M1 Database
	M2 Clock
	M3 Market Providers
	M4 Health Service (app)
	M5 Device Service (app)
Behaviour-Hiding Module	M6 Control
	M7 Constants (server and app)
	M8 Users
	M9 Wallet
	M10 Market
	M11 Activity
	M12 Trading
	M13 Goals†
	M14 Learning†
	M15 Social†
	M16 Scenarios†
	M17 Privacy†
	M18 Navigation (app)
	M19 Screens (app)
	M20 Sync Service (app)
	M21 Reminder Service (app)†
Software Decision Module	M22 Plugins
	M23 JSON Format
	M24 API Service (app)

Table 1: Module Hierarchy

7 Module Decomposition

Modules are decomposed according to the principle of "information hiding". The *Secrets* field in a module decomposition is a brief statement of the design decision hidden by the module. The *Services* field specifies *what* the module will do without documenting *how* to do it. For each module, a suggestion for the implementing software is given under the *Implemented By* title. If the entry is *OS*, this means that the module is provided by the operating system or by standard programming language libraries. *FitFolio* means the module

will be implemented by the FitFolio software.

Only the leaf modules in the hierarchy have to be implemented. If a dash (–) is shown, this means that the module is not a leaf and will not have to be implemented. Server paths under *Implemented By* are relative to `src/backend/app/`.

7.1 Hardware Hiding Modules

Secrets: The interfaces of software and devices the team does not write: the database server, the server's clock, the market data provider, and the phone's health platform and operating system.

Services: Serve as a virtual version of these outside systems for the rest of FitFolio, so that a change to one of them stays inside its module.

Implemented By: –

7.1.1 Database Module (M1)

Secrets: The database server and the connection to it.

Services: Opens sessions to the database, gives each request its own session, and provides the base that every module's tables are defined on. Each module still owns its own tables.

Implemented By: FitFolio server, in `core/db.py`, using SQLAlchemy and PostgreSQL

Type of Module: Abstract Object

7.1.2 Clock Module (M2)

Secrets: The source of the current time.

Services: Gives the current time, with its time zone, to routes and background work. Tests replace it with a fixed time.

Implemented By: FitFolio server, in `core/clock.py`, using the OS clock

Type of Module: Library

7.1.3 Market Providers Module (M3)

Secrets: The market data provider's API.

Services: Gets the latest quote for a stock through one interface that every provider follows, and picks the provider that `MARKET_PROVIDER` names. The Finnhub provider turns Finnhub's replies and errors into FitFolio's own quotes and errors. A fake provider makes up prices for development and tests.

Implemented By: FitFolio server, in `market/providers/`, using Finnhub

Type of Module: Abstract Data Type, with one interface and a Finnhub and a fake implementation

7.2 Behaviour-Hiding Modules

Secrets: The contents of the required behaviours.

Services: Includes programs that provide externally visible behaviour of the system as specified in the software requirements specification (SRS) documents. This module serves as a communication layer between the hardware-hiding module and the software decision module. The programs in this module will need to change if there are changes in the SRS.

Implemented By: –

7.2.1 Control Module (M6)

Secrets: The set of modules that make up the running server.

Services: Builds the server from the list of module plug-ins. It adds each module's routes, and starts and stops each module's background work with the server. It also serves the health check.

Implemented By: FitFolio server, in `main.py`, `registry.py` and `core/routes.py`

Type of Module: Abstract Object

7.2.2 Constants Module (M7)

Secrets: The values of the SRS symbolic constants (SRS Appendix C).

Services: Gives every module the same values, such as the daily currency cap and the price staleness limit, so that a value changes in one place.

Implemented By: FitFolio server, in `core/constants.py`, and a matching file in the app

Type of Module: Library, with exported constants only

7.2.3 Market Module (M10)

Secrets: The live market data: the asset list, the latest prices, how old each price is, and when the market is open.

Services: Keeps every tradable stock's price up to date while the market is open, and marks a price as delayed when it is too old. Gives other modules the assets and their current prices, and serves the asset list and the provider's usage to the phone.

Implemented By: FitFolio server, in `market/`

Type of Module: Abstract Object

7.2.4 Trading Module (M12)

Secrets: The trade rules: buying, selling, holdings and portfolio value.

Services: Buys and sells whole shares at the current price, and refuses a trade that is not allowed, such as one while the market is closed, on a price that is out of date, or without enough currency. Applies each trade request once, and values the user's holdings.

Implemented By: FitFolio server, in `trading/`

Type of Module: Abstract Object

7.3 Software Decision Modules

Secrets: The design decision based on mathematical theorems, physical facts, or programming considerations. The secrets of this module are *not* described in the SRS.

Services: Includes data structure and algorithms used in the system that do not provide direct interaction with the user.

Implemented By: –

7.3.1 Plugins Module (M22)

Secrets: The shape of a module's plug-in.

Services: Defines what a module hands to Control: its name, its routes and its background work. Control can then add any module without knowing anything else about it.

Implemented By: FitFolio server, in `core/plugins.py`

Type of Module: Record

7.3.2 JSON Format Module (M23)

Secrets: The JSON form of amounts and refusals.

Services: Sends amounts to the phone as text with four decimals, so no precision is lost, and sends every refusal in the same shape: a code and a message.

Implemented By: FitFolio server, in `core/money.py` and `core/errors.py`

Type of Module: Library

8 Traceability Matrix

This section shows two traceability matrices: between the modules and the requirements and between the modules and the anticipated changes.

Req.	Modules
R1	M??, M??, M??, M6
R2	M??, M??
R3	M??
R4	M??, M6
R5	M??, M??, M6, M??, M??, M??
R6	M??, M??, M6, M??, M??, M??
R7	M??, M??, M6, M??, M??
R8	M??, M??, M6, M??, M??
R9	M??
R10	M??, M??, M6
R11	M??, M??, M??, M6

Table 2: Trace Between Requirements and Modules

AC	Modules
AC1	M??
AC2	M??
AC??	M??
AC??	M??
AC??	M??
AC??	M??
AC??	M??
AC??	M??
AC??	M6
AC??	M??
AC??	M??
AC??	M??

Table 3: Trace Between Anticipated Changes and Modules

9 Use Hierarchy Between Modules

In this section, the uses hierarchy between modules is provided. Of two programs A and B, A *uses* B if correct execution of B may be necessary for A to complete the task described in its specification. That is, A *uses* B if there exist situations in which the correct functioning of A depends upon the availability of a correct implementation of B. Figure 1 illustrates the use relation between the modules. It can be seen that the graph is a directed acyclic graph (DAG). Each level of the hierarchy offers a testable and usable subset of the system, and modules in the higher level of the hierarchy are essentially simpler because they use modules from the lower levels.

An arrow from module A to module B means that A uses B: the code of A imports B. To keep Figure 1 readable, one arrow into a group stands for uses of the modules in that group. The shared group holds the modules that many others use. For example, every server module with routes uses M8 Users to find the current user. Modules in the shared group can also use each other. Table 4 lists every use.

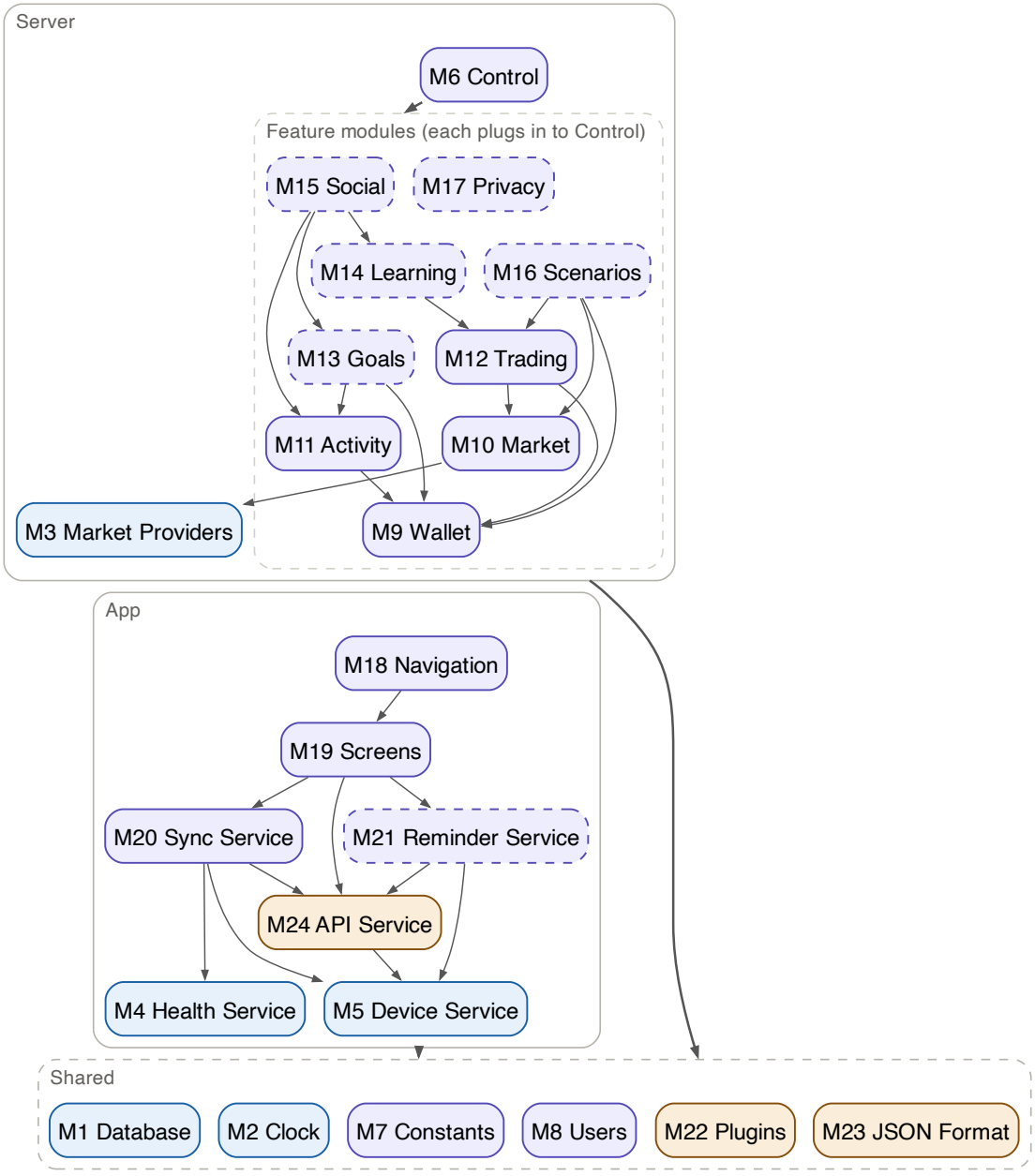


Figure 1: Use hierarchy among modules. Blue: hardware-hiding. Purple: behaviour-hiding. Orange: software decision. A dashed outline marks a module built after the proof of concept.

Module	Uses
M1 Database	None
M2 Clock	None
M3 Market Providers	M2
M4 Health Service	None
M5 Device Service	None
M6 Control	M1, M8, M9, M10, M11, M12, M13, M14, M15, M16, M17, M22, M23
M7 Constants	None
M8 Users	M1, M2, M7, M22, M23
M9 Wallet	M1, M8, M22, M23
M10 Market	M1, M2, M3, M7, M22, M23
M11 Activity	M1, M2, M7, M8, M9, M22, M23
M12 Trading	M1, M2, M8, M9, M10, M22, M23
M13 Goals	M1, M2, M7, M8, M9, M11, M22, M23
M14 Learning	M1, M7, M8, M12, M22, M23
M15 Social	M1, M2, M8, M11, M13, M14, M22, M23
M16 Scenarios	M1, M7, M8, M9, M10, M12, M22, M23
M17 Privacy	M1, M8, M22, M23
M18 Navigation	M19
M19 Screens	M7, M20, M21, M24
M20 Sync Service	M4, M5, M7, M24
M21 Reminder Service	M5, M7, M24
M22 Plugins	None
M23 JSON Format	None
M24 API Service	M5

Table 4: Modules each module uses

10 User Interfaces

[Design of user interface for software and hardware. Attach an appendix if needed. Drawings, Sketches, Figma —SS]

11 Design of Communication Protocols

[If there is communication between hardware and software, it may be necessary to explain the design of the communication protocol. This section should be filled in with NA if it is not appropriate for a given project. —SS]

12 Database Design

[If a database is a significant component of the project, the database design should be summarized here. This section should be filled in with NA if it is not appropriate for a given project. —SS]

13 Timeline

[Schedule of tasks and who is responsible —SS]

[You can point to GitHub if this information is included there —SS]

14 Generative AI Use Disclosure

[ChatGPT (version 5) was used to brainstorm ideas for the template for this disclose section. —SS]

14.1 AI Tools Used

[Give the name of the AI tool(s) used and the version. —SS]

14.2 How and Where Used

[Explain what the tool(s) were used for. Examples include: brainstorming, explaining a technical concept, reviewing the document, generating or modifying text, generating or modifying diagrams, identifying errors or omissions or inconsistencies. —SS]

[For each of the uses, identify which parts of the document were affected. —SS]

[You do not need to report every interaction, but you should disclose the uses that materially contributed to your work —SS]

14.3 Verification of AI-Generated Content

[

- Describe how the team checked AI-generated or AI-assisted material before including it in the document.

- In particular, verify factual claims, technical assertions, requirements, calculations, references, and other information for which an AI system may produce incorrect or fabricated results.
- **The team is responsible for the accuracy, completeness, consistency, and appropriateness of the submitted document, regardless of whether material was generated by a human or an AI system.**

—SS]